

Tomato Sorter v2.0

AI-Powered Tomato Sorting System

Raspberry Pi 5 | Arduino Uno | YOLO11n | Flask

PROTOTYPE DOCUMENTATION

Laurence De Guzman

Technological University of the Philippines, Manila

April 2026

1. Executive Summary

This document describes the design and implementation of the Tomato Sorter v2.0, an automated tomato classification and sorting system developed as a thesis prototype. The system uses a YOLO11n deep learning model running on a Raspberry Pi 5 to classify tomatoes by ripeness condition (ripe, unripe, or rotten) and physically sort them into the appropriate storage bins through a conveyor and servo mechanism controlled by an Arduino Uno. A real-time web dashboard running in kiosk mode on a 7-inch touchscreen display provides live monitoring, environmental data from dual DHT22 sensors, and manual control of the system.

The system is designed to operate fully offline (no internet required), auto-launch on boot, and survive component failures through systemd service auto-restart. From power-on to fully operational dashboard takes approximately 30 seconds.

2. System Architecture

The system is built on a 5-layer software architecture with strict separation of concerns. Each layer communicates only with the layer directly below it, or via an in-memory event bus, ensuring components remain testable and replaceable in isolation.

Layer	Module	Responsibility
Layer 4	api/	Web interface (Flask + WebSocket)
Layer 3	app/	Orchestration, state, event bus
Layer 2	services/	Stateless workers (detector, sensors, classifier)
Layer 1	hardware/	Pure I/O wrappers (camera, GPIO, DHT22)
Layer 5	persistence/	SQLite database, CSV exporter

Hardware Control Split

Compute and real-time control responsibilities are split between two boards. The Raspberry Pi 5 handles AI inference, the dashboard, logging, and high-level orchestration. The Arduino Uno handles real-time hardware control where microsecond precision is required, such as servo PWM and IR sensor interrupt handling. The two boards communicate over USB serial at 9600 baud through the device file `/dev/ttyACM0`.

Raspberry Pi 5 handles	Arduino Uno handles
Camera capture + YOLO inference	Servo 1 (gate) — hardware PWM
Flask web dashboard	Servo 2 (sorter) — hardware PWM
SQLite event logging	IR sensor (interrupt-driven)
DHT22 temperature/humidity	Relay 1 / Relay 2 (fan control)
BTS7960 conveyor PWM	Serial command parsing

Raspberry Pi 5 handles	Arduino Uno handles
WebSocket live updates	

3. Materials and Components

The complete bill of materials for the prototype is organized below by functional category.

3.1 Main Processing Unit

Component	Specification	Qty
Raspberry Pi 5	8GB RAM, Debian 13 Trixie OS	1
7-inch DSI Touchscreen	DSI cable + 5V/GND on pins 4 and 6	1
MicroSD Card	32GB	1
Pi 5 USB-C Power Adapter	5V / 5A official	1
Active Cooling Fan	For sustained CPU inference	1

3.2 Microcontroller

Component	Specification	Qty
Arduino Uno	ATmega328P, hardware PWM control	1
USB-A to USB-B Cable	Pi-Arduino link (power + serial)	1

3.3 Vision and AI

Component	Specification	Qty
USB Camera	640x480, /dev/video0	1
YOLO11n Model	NCNN format, 3 classes, 480x480 input	1

3.4 Actuation

Component	Specification	Qty
Servo Motor (Gate)	Releases tomatoes one at a time	1
Servo Motor (Sorter)	180 deg — left=ripe, center=unripe, right=rotten	1
BTS7960 Motor Driver	H-bridge for 12V DC conveyor motor	1
DC Conveyor Motor	12V DC	1

3.5 Sensing

Component	Specification	Qty
DHT22 Sensor	Temperature/humidity — Bin 1 (ripe)	1
DHT22 Sensor	Temperature/humidity — Bin 2 (unripe)	1

Component	Specification	Qty
IR Sensor	Detects tomato at sort point	1

3.6 Power Supplies

Component	Specification	Qty
12V Power Supply	Conveyor motor + fans (via relay)	1
5V Power Supply	2A min — Servo 1 and Servo 2	1

3.7 Relay and Cooling

Component	Specification	Qty
Dual Relay Module	Controls Fan 1 and Fan 2	1
DC Fan	Bin 1 ventilation (ripe)	1
DC Fan	Bin 2 ventilation (unripe)	1

3.8 Wiring and Prototyping

Component	Specification	Qty
Breadboard	Full-size	3
Jumper Wires	Assorted M-M, M-F, F-F	1 set

3.9 Storage Bins

Bin	Class	DHT22 Sensor	Cooling Fan
Bin 1	Ripe	Yes	Yes (Relay 1)
Bin 2	Unripe	Yes	Yes (Relay 2)
Bin 3	Rotten	No	No

4. Sorting Workflow

The complete operational cycle of the system is shown below. The key safety rule is that only one tomato is allowed on the conveyor at any time. Servo 1 only opens the gate after Servo 2 has finished sorting the previous tomato.

- Servo 1 (gate) opens briefly
 - One tomato falls onto the conveyor
 - Servo 1 closes quickly
 - Conveyor runs (12V motor via BTS7960)
 - USB camera captures tomato
 - YOLO11n classifies as ripe / unripe / rotten
 - Detection logged to SQLite database
 - IR sensor at end of conveyor detects tomato arrival
 - Pi commands Arduino to rotate Servo 2:

Left = Ripe -> Bin 1

Center = Unripe -> Bin 2

Right = Rotten -> Bin 3
 - Servo 2 holds for ~600ms (tomato falls into bin)
 - Servo 2 returns to home position
 - Servo 1 opens again -> next tomato -> repeat

If the camera fails to detect a tomato (poor lighting, occlusion, or speed mismatch), the system applies a safe fallback: the tomato is sorted as 'unripe' to prevent good produce from being discarded.

5. Pi-Arduino Communication Protocol

Serial communication operates at 9600 baud over USB. Commands are newline-terminated ASCII strings. The Pi initiates most exchanges; the Arduino additionally publishes asynchronous events when the IR sensor triggers.

Pi to Arduino (Commands)

Command	Action
SERVO1:OPEN	Gate opens briefly
SERVO1:CLOSE	Gate closes
SERVO2:LEFT	Sort to ripe bin
SERVO2:CENTER	Sort to unripe bin
SERVO2:RIGHT	Sort to rotten bin
RELAY1:ON / OFF	Fan 1 control
RELAY2:ON / OFF	Fan 2 control

Arduino to Pi (Events)

Message	Meaning
IR:TRIGGERED	Tomato detected at sort point
SERVO1:DONE	Gate movement complete
SERVO2:DONE	Sorter movement complete
OK	Generic acknowledgement

6. Software Stack

Software	Purpose
Python 3.13	Main language on the Pi
OpenCV (system, GTK/Qt5)	Camera capture and image preprocessing
NCNN	YOLO inference (ARM-optimized, 2-3x faster than PyTorch)
Flask + Flask-SocketIO	Web server and real-time WebSocket updates
SQLite	Detection, sensor, and event logging
gpiozero + I2C	Pi 5 GPIO control (RP1 chip compatible)
adafruit-circuitpython-dht	DHT22 sensor reading
pyserial	Pi to Arduino serial communication
Arduino IDE / C++	Arduino Uno firmware
systemd	Service auto-start and auto-restart on boot
Chromium	Kiosk-mode dashboard display

7. Power Distribution

The system uses three independent power sources, all sharing a common ground reference. This isolation prevents brown-outs on the Pi caused by inrush current from servos and motors.

Source	Powers
Pi USB-C Adapter (5V/5A)	Raspberry Pi 5 only
12V Power Supply	Conveyor (via BTS7960) + fans (via relay load side)
5V Power Supply (2A+)	Servo 1 and Servo 2
Pi 5V GPIO pin	DHT22 sensors, IR sensor logic, relay coil VCC
Pi USB to Arduino	Powers Arduino + provides serial link

8. Dashboard Interface

The dashboard runs in Chromium kiosk mode on the 7-inch DSI display at `http://localhost:5000`. The interface follows an industrial SCADA design pattern: clean, neutral dark theme, no decorative animations, optimized for unattended operation.

Dashboard Features

- Live camera feed with detection bounding boxes overlaid in real-time
- Bin counters: Bin 1 (Ripe), Bin 2 (Unripe), Bin 3 (Rotten)

- Environmental gauges: temperature and humidity per bin
- System status: conveyor, fans, servo positions
- Manual fan toggles and servo calibration sliders
- Activity log: last 10 detections with timestamp and confidence
- WebSocket push updates approximately every 500 ms

9. Reliability and Risk Mitigation

Risk	Mitigation
DHT22 read failures (40-60% normal)	Caching layer; UI shows real vs cached badge
Pi CPU overheating during long runs	Active cooler + thermal alarm in dashboard
Service crash mid-operation	systemd auto-restart on failure
Camera misses detection	Fallback class = unripe (safe default)
Network failure on demo day	Fully local — no internet required
SD card corruption	Regular DB backups; NVMe SSD recommended

10. Boot Sequence

When the Raspberry Pi is powered on, the following sequence runs automatically without user intervention:

1. Debian Trixie boots and auto-logs in as user 'bacadasa'

2. systemd starts three services in order:

- tomato-sensors.service (DHT22 polling loop)

- tomato-detector.service (Camera + YOLO inference)

- tomato-dashboard.service (Flask web server)

3. Wayfire compositor launches the desktop

4. Chromium opens fullscreen at http://localhost:5000

5. Dashboard is fully operational

Total boot-to-dashboard time: approximately 30 seconds.

11. Conclusion

Tomato Sorter v2.0 demonstrates a panel-grade integration of edge AI inference, real-time microcontroller hardware control, and a production-quality web dashboard on consumer-grade hardware. The 5-layer software architecture ensures each subsystem can be tested, replaced, or extended independently. The split between the Raspberry Pi and the Arduino provides the best of both worlds: high-level Linux services for AI and web, and microcontroller precision for real-time actuation.

All configuration is externalized to YAML, all events are logged to SQLite for reproducible analysis, and the system auto-launches in kiosk mode for unattended operation. The result is a system that is not only functional but defensible to a thesis review panel as an example of clean engineering practice on embedded hardware.